

实验 8：流水线 CPU 设计与测试

一、实验目的

1. 掌握基础流水线 CPU 的设计方法。
2. 掌握基础流水线 CPU 中阻塞检测和旁路转发的实现方法。
3. 掌握基础流水线 CPU 分支静态预测实现方法。
4. 掌握基础流水线 CPU 程序代码优化的编程方法。
5. 掌握基础流水线 CPU 硬件设计优化的设计方法。

二、实验环境

Logisim: <https://github.com/Logisim-Ita/Logisim>

RISC-V 模拟器工具 RARS: <https://github.com/thethirdone/rars>

三、实验原理

单周期处理器的指令执行采用串行方式，CPU 总是在执行完一条指令后才取出下条指令执行。这种串行方式没有充分利用执行部件的并行性，因而指令执行效率低。指令的执行也可以采用流水线方式，将指令执行过程分成多个功能模块，各个模块运行实现流水线化，重叠执行多条指令，以提高 CPU 执行指令的效率，从而达到高性能的计算。

本实验在 RV32I 单周期处理器的基础上进行改进，实现经典的五段流水线式 CPU 设计。通过五段流水线设计理解并掌握流水线处理器中数据通路和流水段寄存器设计方法，学习结构冒险、数据冒险以及控制冒险的基本解决方案。

单周期处理器架构存在下列一些问题：

1、每条指令的所有任务均需要在一个周期内完成，较复杂的指令需要经过多个步骤和多个逻辑部件，且这些步骤无法并行。

2、单周期处理器的各个部件采用紧耦合的方式实现，修改一个部件的设计将会影响许多关联的部件。这与现代数字系统设计中需要遵循的模块化、松耦合的指导思想相违背。

为了解决上述问题，在设计 CPU 的时候可以采用流水线架构来实现。在流水线架构中，单条指令的操作被划分为多个相对独立的步骤。处理器中含多个流水段，每个流水段只完成指令中相对独立的一项操作。在流水线上就可以同时执行多条指令，每个指令所处的执行阶段不同，在同一个时刻由不同的流水段进行处理。当一条指令顺序通过所有的流水段之后就完成了该指令的执行。流水线的主要优势在于：单个流水段的操作简单，需要的执行时间较短，因而可以在较高的主频上运行。同时，每个时钟周期近乎完成一条指令，

指令执行的吞吐量提高。

流水线处理器中单个指令需要顺序经过多个流水段，指令的执行时长不一定比单周期处理器短。但是，由于多条指令在流水线上实现了空间并行，流水线处理器可以每个周期完成一条指令的执行，且时钟周期较单周期处理器短。所以，流水线处理器的指令吞吐率较单周期处理器高。

1、流水线 CPU 设计原理

流水线设计的原则是：指令流水段个数以最复杂指令所需的功能段个数为基准；流水段的时钟周期以最复杂流水段的操作所用时间为单位。一条指令的执行过程可被分成若干个阶段，每个阶段由相应的功能部件完成。首先要对每条指令的执行过程进行分析，以确定流水线每个功能段的功能以及执行时间。通常 RISC-V 指令的执行过程可以划分为如下 5 个功能段。

取指令段 IF (Instruction Fetch)：主要完成从指令存储器中读取指令的工作，并计算下条指令的地址。

指令译码段 ID (Instruction Decode)：主要完成指令译码、立即数扩展、读取源寄存器操作数，并根据指令生成控制信号。

执行段 EX (Execution)：主要完成 ALU 运算，以及计算转移目标地址。

访存段 M (Memory access)：主要完成对数据存储器的读写操作，以及检测分支转移指令是否需要跳转等操作。

写回段 WB (Write Back)：主要完成将执行结果写回到寄存器中的操作。

每条指令取指 IF 和译码 ID 这两个功能段都一样，后面的功能段随不同类型指令而不同，有些指令需要通过加入“空”功能段来构成 5 个功能段。在插入“空”段时，应遵循以下两个原则：①每个功能部件每条指令只能用一次（如寄存器写口不能用两次或以上）；②每个功能部件必须在相同的阶段被使用（如寄存器写口总是在第 5 阶段被使用）。

因此，R-型指令、I-型运算类指令、U-型指令和 J-型指令需在 WB 之前加一个空的 M 段，使得其 WB 段和 load 指令的 WB 对齐，都在第 5 段；S-型指令和 B-型指令在第 4 个功能段后加一个空的 WB 段。这样，所有指令都有 5 个功能段。

指令执行需经历 5 个流水段：IF、ID、EX、M 和 WB，每个流水段都在不同的功能部件中执行，相邻流水段之间通过流水段寄存器来存储和传递数据以及控制信号。例如，IF/ID 寄存器是介于 IF 段和 ID 段之间的寄存器。由于相邻流水段段间传递的信息不一样，所以各流水段寄存器的长度也不一样。五个流水段的间隔设计了四个流水段寄存器，对应简称为 IF/ID、ID/EX、EX/M、M/WB 流水段寄存器。

时序分析

流水线 CPU 的时序设计相对于单周期处理器要简单，在本实验的设计中将时钟的下降沿作为周期的起点和终点。即每个阶段在时钟下降沿开始，利用流水段寄存器中的信息执行本流水段的操作，然后在下一个时钟下降沿将结果写入下一阶段的流水段寄存器，将工作交给下一个流水段。

在流水线处理器中，PC 寄存器在 IF 流水段中，使用时钟下降沿来更新，即每次时钟下降沿时 PC 刷新，准备取下一条指令。指令存储器包含在 IF 流水段中，采用异步读取指令的方式，即 IF 阶段在下降沿更新 PC 之后，随后读取指令，并在下一个下降沿写入 IF/ID 流水段寄存器。

寄存器堆在 ID 流水段中，寄存器的读取不受时钟沿控制，只要读端口编号改变，读取结果就立即改变。寄存器的写入采用上升沿写入的方式，写入的寄存器号、数据及写使能信号均由 WB 阶段提供。这里写入采用提前半周期的上升沿写入的主要原因是允许 WB 阶段向 ID 阶段转发数据，主要考虑数据冒险的处理。

数据存储器在 M 阶段中，采用下降沿写入数据，读取数据采用异步或者上升沿读取的方式。在上升沿读取数据的主要原因是为了保证在 M 阶段执行的相关数据信号已经保持稳定。

总之，除了寄存器堆是时钟信号上升沿写入外，其他时序部件均下降沿写入。

2、流水线冒险处理

在 CPU 中指令的执行存在依赖关系的，后续指令的执行依赖于前序指令的执行结果。因此，在流水线 CPU 设计中需要处理指令之间的三大类依赖关系，对应流水线中的结构冒险、数据冒险和控制冒险。

结构冒险是指不同执行阶段中的指令需要使用同一个硬件资源，造成硬件资源冲突。例如，当两条指令同时要写入寄存器，且寄存器一个周期只允许一个写入时，就存在结构冒险。在五段流水线结构中，通过插入空流水段，使得每个阶段的硬件资源都只有一个指令在使用；在寄存器堆中读写操作分时序执行，相当于两个独立部件；设计独立的下地址计算加法器等方式解决了结构冒险。

数据冒险是指指令执行过程中，前后指令对寄存器的读写产生冲突，造成需要转发或阻塞部分指令等待数据冲突消失。简单五段流水线只会出现 Read After Write (RAW) 冒险，RAW 类冒险是指后续指令需要使用前面指令的计算结果，但是前序指令还未将数据写入寄存器时，后续指令就需要读取寄存器了，此时后续指令需要阻塞，或通过旁路转发的方式获取前序指令的执行结果。

解决数据冒险有两种基本方案：旁路转发和阻塞。

运算类指令的执行结果在 EX 段的结束时就已经确定，但是该结果需要流过 M 和 WB 两个阶段才能写入寄存器堆。因此，可以在 EX 段时进行转发检测，当发现当前指令的操作数需要前序指令的执行结果时，将前序指令的运算结果通过旁路转发方式送到后续指令的 EX 段，让后续指令能立刻用上最新的结果。数据转发的优先级：如果 EX 阶段的寄存器号与多个后续阶段的目的寄存器均匹配时，离 EX 段指令越近的数据是越新的，因此，越靠近的指令转发数据优先级越高。

由于 Load 指令需要等待到 M 阶段才能从数据存储器中读取出结果，如果 Load 指令后紧跟一条需要使用 Load 结果的指令，则会发生 Load-use 冒险。在这种情况下，当前序 Load 指令处于 M 阶段读取数据存储器的时候，后续指令已经在 EX 阶段了，但是新数据还没准备好。此时需要阻塞后续指令流水段的执行。

阻塞是指暂停某个流水段的执行。当流水线中某个阶段暂时无法执行时，该阶段前面的所有流水段都应该停下来等待其执行完毕再继续执行。暂停流水段的执行通常通过保持流水段寄存器内容不变来实现，可将 PC 寄存器和相关流水段寄存器的写使能信号置于无效状态。

可在 ID 段进行阻塞检测，检测相邻指令间是否存在 Load-use 冒险。如果存在，则要阻塞 IF 和 ID 流水段的执行，将 PC、IF/ID 流水段寄存器的写使能信号置于无效状态，这两个流水段暂停执行，其他流水

段不受影响进行执行；为了防止当前指令产生错误的输出，需将 ID 段中指令的控制信号全部清零。IF 和 ID 流水段将暂停执行一个时钟周期，等待前序 Load 指令进入 WB 阶段后，通过旁路转发获取数据存储器读取的数据。

控制冒险是指常情况下流水线中正按顺序执行指令时，当遇到分支、跳转等指令可能改变指令执行顺序时，导致流水段的指令被错误执行，造成取指错误，在跳转目标地址时需要将流水线中错误的指令冲刷掉。对于控制冒险，可以通过硬件阻塞或插入空指令等方法来解决；对于分支指令带来的分支冒险，可以通过分支预测来降低由于分支冒险带来的时间损失。在本实验中通过静态预测来解决分支和跳转指令带来的控制冒险，静态预测总是预测不满足，流水线按顺序继续执行分支指令的后续指令。如果在数据通路中检测到实际条件确实不满足时，则预测正确，没有任何时间损失；如果检测到实际条件满足时，则预测不正确，此时，需要将分支指令的后续不该被执行指令进行冲刷操作，并跳转到转移地址来继续执行程序。

冲刷是指将流水线中取到或执行了错误指令的清除掉，变成空操作或者气泡（NOP 或 Bubble）。在本实验的设计中，为了简化处理，将跳转检测模块放在了 M 阶段。由于 M 阶段与 IF 阶段间隔三个流水段，因此每次预测错误将会冲刷 3 个流水段的指令，相当于引入三个空操作，可通过将相关流水段寄存器的清零信号置为有效状态来实现。

3、流水线 CPU 数据通路设计

根据对 RV32I 中 37 条目标指令的分析，流水线 CPU 的数据通路的设计如下。

1. 取指令段 IF

取指阶段（IF）的主要功能是：1）确定 PC，并从指令存储器中取出 PC 地址的对应指令。2）时钟下降沿时更新 PC。PC 的来源有三个：

顺序执行：使用加法器实现 $PC+4$ ；

跳转执行：当需要进行分支或跳转时，使用在 EX/M 流水段中的转移地址 PCTarget 更新 PC；在 M 段中进行跳转检测，生成跳转信号 PCsrc，当 PCsrc 有效时表示需要跳转；

初始地址：当复位信号 Reset 有效时，使用初始地址 Init Addre 来更新 PC。

在下降沿到来后，PC 更新到新值，新的 PC 送给指令存储器的读地址输出指令 Instr。

当阻塞信号或程序中止信号有效时，需暂停 IF 阶段的执行，PC 保持原值不变。

当跳转信号与阻塞信号 Stall 或中止信号 Halt 都有效时，跳转信号的优先级最高。

由于指令在 IF 阶段获取的信息只有 PC 和 32 位的指令 Instr，在 IF/ID 流水段寄存器中传递 PC 和 Instr。

2. 指令译码段 ID

译码阶段 ID 流水段的功能是：1）指令字段解析；2）对指令中的操作码和功能码进行译码，生成相应的控制信号；3）通过扩展器对指令中的立即数字段进行扩展后生成的完整 32 位立即数；4）根据指令中 rs1 和 rs2 字段的值读取寄存器堆的相应寄存器内容 BusA 和 BusB。

解析指令的方式与单周期基本相同，从指令中获取出立即数 imm、rs1、rs2、rd 等字段信息。控制器

的设计与单周期 CPU 控制器的设计类似，根据操作码和功能码来生成控制信号。控制信号的定义与单周期相同，其中 ExtOp 用于确定立即数扩展方式，在本流水段使用。在 EX 阶段使用的控制信号是 BandJ、ALUAsrc、ALUBsrc 及 ALUctr。在 M 阶段使用的控制信号包括 MemOp 及 MemWr。在 WB 阶段使用的控制信号是 RegWr、MemtoReg 和 Halt。这些控制信号将随着指令执行中间结果一起流过各个流水段，在对应的流水阶段控制执行部件完成指令对应的操作。每条指令在每一个阶段执行过程中所需要的数据及控制信息均通过流水段寄存器中对应字段直接获取。

程序中止信号 Halt 有效时表示当前指令为中止指令，需要将 PC 寄存器暂停更新，需等待流水线中的所有指令执行结束后才中止程序执行，因此需要传递 Halt 信号到 WB 段，停止当前的程序执行。

在完成了译码阶段的工作后，ID 阶段需要将指令执行的所有信息写入 ID/EX 流水段寄存器。具体写入内容包括数据和控制信号两部分，其中数据信息包括 PC、imm、rd、rs1、rs2、BusA、BusB。

在当前流水段执行阻塞检测，当阻塞信号有效时，PC 和 IF 流水段寄存器保持不变，当前指令的控制信号全部清零。

3. 执行段 EX

执行阶段（EX）的主要功能是 1）利用 ALU 进行数据运算；2）计算分支指令和跳转转移地址。不同指令经 ID 段译码后得到不同的控制信号，用来控制执行部件进行不同的操作。

ALU 运算过程类似单周期处理器，先根据 ALUAsrc、ALUBsrc 来确定 ALU 的两个输入，然后根据 ALUctr 执行相应的运算获取运算结果 Result 及 Zero 标志位。跳转地址的运算也类似单周期，这里的主要不同点是流水线处理器在 IF 阶段就直接算好了 PC+4，因此在执行阶段主要根据分支控制信号 BandJ 来选择 PC 或 BusA，加上立即数来计算跳转目的地址。

在当前阶段进行旁路转发检测，当源寄存器好与 M 段的 rd 或 WB 段的 rd 相同，且非 0 号寄存器时，需将 M 段或 WB 段的运算结果转发到 ALU 的输入端。

在执行阶段完成后，EX/M 流水段寄存器需要传递的数据主要包括 Result、零标志位 Zero、转移地址 PCTarget、ALU 操作数 B（经过转发检测选择后的 ALU 操作数 B）和 rd。需要传递的控制信号包括 M 段控制信号及 WB 段控制信号。

4. 访存段 M

访存阶段（M）的主要功能是 1）对数据存储器进行读写；2）分支跳转判断。

数据存储器的读写流程与单周期类似，都是将地址信号连接到数据存储器的 addr 端口，并将需要写入的数据放在存储器的 DIn 端口，根据 MEM 阶段的控制信号执行读写操作即可。在时钟的下降沿时写入。

跳转检测与单周期类似，根据 BandJ 判断跳转的类型，然后根据 ALU 输出结果最低位 Result[0]和零标志位 Zero 来确定是否需要跳转，最后输出跳转信号 PCsrc。当 PCsrc 有效时，表示需要进行跳转，在 IF 段使用转移地址 PCTarget 来更新 PC，同时对 IF/ID、ID/EX、EX/M 三个流水段寄存器进行冲刷，清除错误指令的执行信息。

M 阶段完成后，M/WB 流水段寄存器需要将 Result、数据存储器的输出 Dout 和 rd 传给下一阶段。同时，也要将 WB 阶段的控制信号传给 WB 阶段。

5. 写回段 WB

写回阶段（WB）主要功能是将最终运算结果写回到目的寄存器中，其执行的动作比较简单。首先，根据 MemtoReg 选择是将 Result 还是 Dout 写回，然后根据 RegWr 来决定是否要写回。将寄存器堆的写入触发设置在时钟的上升沿，这样可以实现将 WB 阶段需要写入寄存器的数据直接转发到 ID 阶段。

完成把数据写入寄存器堆的功能，当前指令的执行结束；如果 Halt 信号有效，则结束程序的执行。

综上所述，每个流水段寄存器中保存的信息包括两类：一类是后面阶段需要用到的所有数据信息，包括 PC、立即数、目的寄存器、ALU 运算结果、标志信息等，它们是前面阶段在数据通路中执行的结果；还有一类是前面传递过来的后面各阶段要用到的所有控制信号。

在流水线处理器中，控制信号一旦在 ID 段由控制器生成就不会改变，并和数据信息同步地依次传递到后面的流水段中。流水线控制器的设计可以完全按照单周期控制器设计的思路进行。

需要注意的是名称相同的信号可能在不同流水段都需要使用，建议在信号命名时加上流水段的前缀。例如，用 rd 来表示指令的目的寄存器时，该信息有可能随着指令流过不同的流水段，而且当多条指令同时在不同阶段时，每个流水段对应的 rd 很可能不同。可以将 ID 阶段的 rd 命名为 ID.rd，EX 阶段的 rd 命名为 EX.rd，这样代码会相对更加清晰可读。

流水线 CPU 的设计原理图如图 8-1 所示。

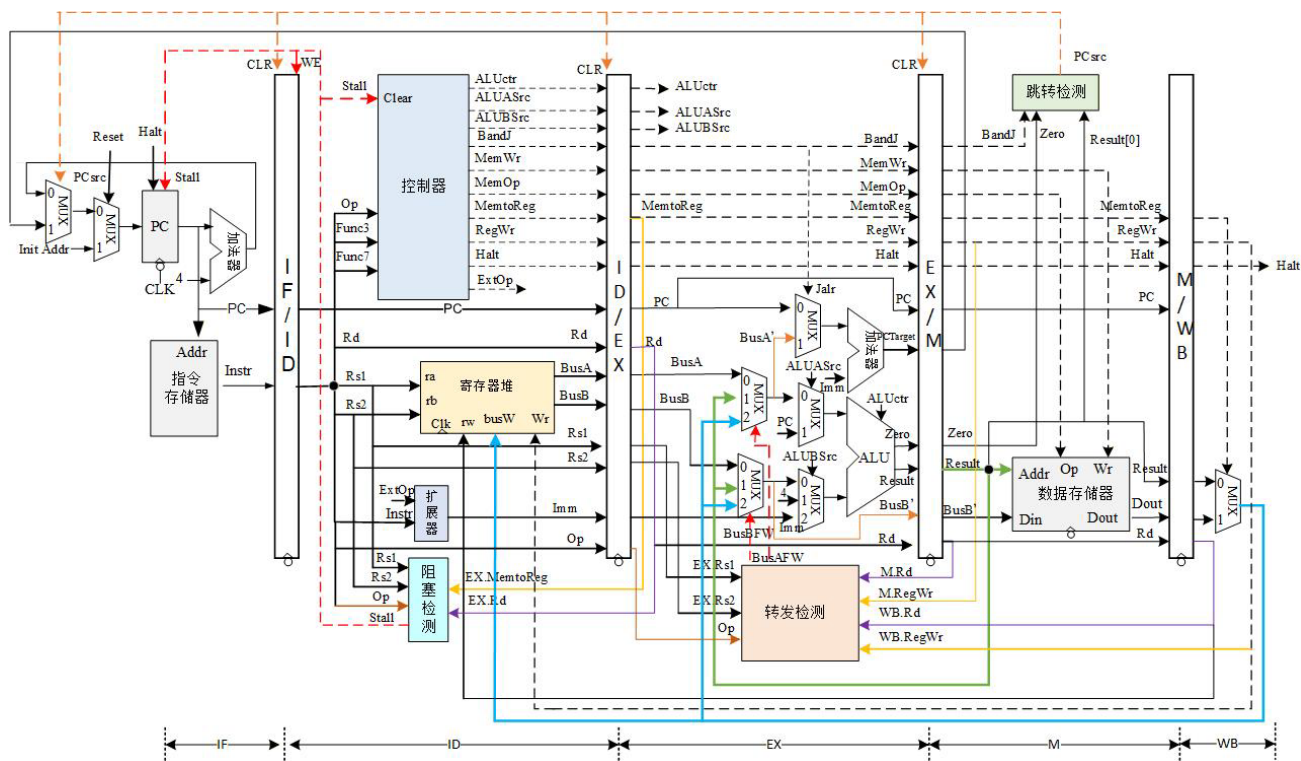


图 8-1 流水线 CPU 设计原理图

四、实验内容

1、流水线 CPU 设计实验

4.1 转发检测模块设计

转发检测一般在执行 EX 段生成转发信号。当 EX 段中检测到源寄存器号与 M 段、WB 段的目的地寄存器号相同，且不是 0 号寄存器，指令类型为运算类指令（RegWr=1）时，则生成转发选择信号。

对于 BusA 转发检测信号 BusAFW，当 BusAFW=00 时，选择寄存器堆的输出 BusA，无须转发；当 BusAFW=01 时，转发来自 EX/M 流水段寄存器运算结果 M.Result；当 BusAFW=10 时，转发来自 WB 段 MemtoReg 选择器结果 WB.Result；生成 BusAFW 的逻辑表达式。

对于 BusB 转发检测信号 BusBFW，判断方法类似。

需要考虑转发数据的优先级，当 M.rd 与 WB.rd 都和 EX 段源寄存器号相同时，离当前指令更近的数据优先级更高。

根据图 8.2 所示的引脚布局图来设计转发检测模块。

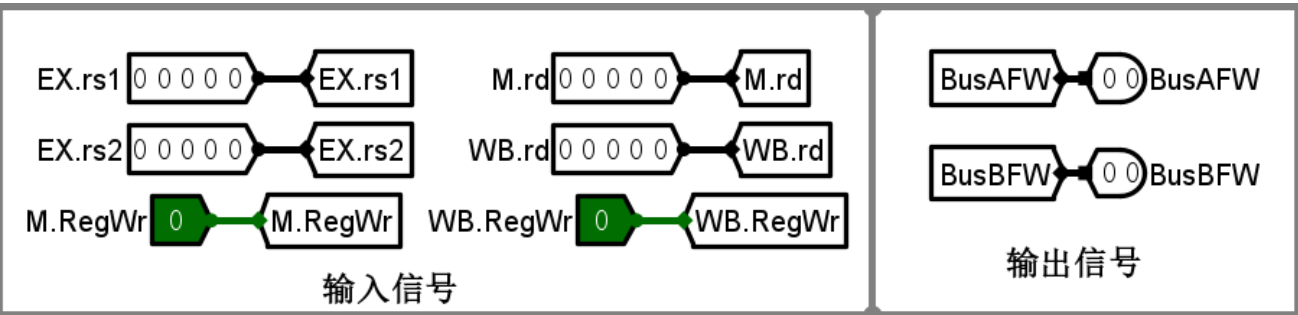


图 8.2 转发检测模块引脚布局图

4.2 阻塞检测模块设计

阻塞检测在 ID 段执行，用于解决 load-use 冒险，需要检测前序指令为 load 指令，当前指令的源寄存器地址与前序指令的目的寄存器地址相同。Load 类型指令的控制信号 MemtoReg 为 1，可通过判断前序指令的该信号是否为 1 来判断是否为 load 指令。判断当前指令是否包含源寄存器 rs1 或 rs2 则需要通过指令类型来判断。R、I、S 和 B 四种指令中包含源寄存器 rs1，R、S 和 B 三种指令中包含源寄存器 rs2。可通过 opcode 来识别指令类型。

当阻塞信号有效时，PC 和 IF/ID 流水段寄存器写使能信号无效，ID 段中的指令控制器输出信号清零，ID 段的当前指令的执行暂停一个时钟周期。

根据图 8.3 所示的引脚布局图来设计阻塞检测模块。

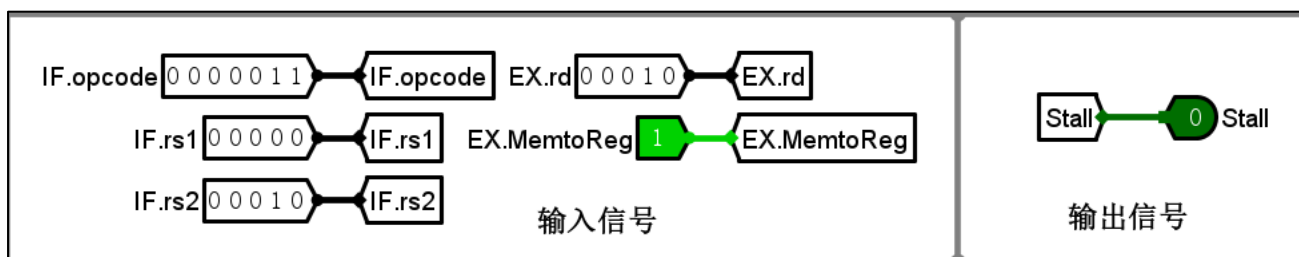


图 8.3 阻塞检测模块引脚布局图

4.3 跳转检测模块设计

跳转检测与单周期 CPU 设计的跳转控制器模块类似，输入信号为 EX.BandJ、EX.Result[0]和零标志信号 EX.Zero，输出信号为 PCsrc，高电平有效。当 PCsrc=1 时，表示预测失败，需跳转到转移地址后继续执行程序，同时冲刷前面流水段中的指令信息。本实验中在 M 段进行分支跳转检测，当预测不正确，需要将前面按顺序读取后续指令在 IF、ID 和 EX 三个流水段中读取错误的指令执行信息清除掉，流水线相当于执行了三条空指令 NOP。

根据图 8.4 所示的引脚布局图来设计跳转检测模块。

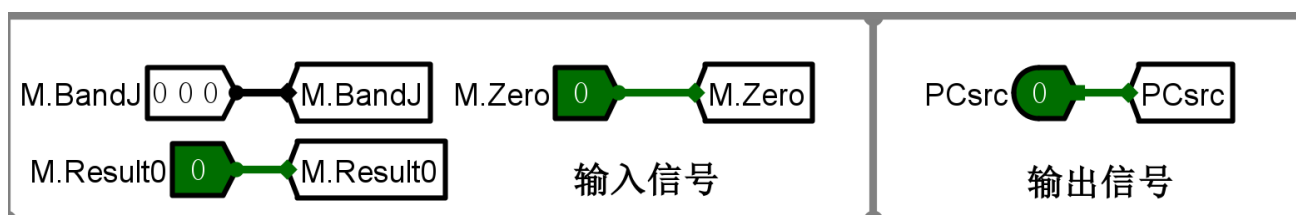


图 8.4 跳转检测模块引脚布局图

实际上 RISC-V 的空指令一般是 `addi zero,zero,0`，对应二进制是 `0x00000013`。在实现过程中也可将全零指令也译码为空指令。

4.4 流水段寄存器设计

流水段寄存器用于在相连的流水段之间传递数据和控制信号。流水段寄存器中除了必须包含需要传递的数据和控制信息外，作为时序电路还需要包含时钟信号 CLK、清零信号 CLR 和写使能信号 WE。且根据时序设计在时钟信号下降沿时写入数据。当阻塞信号或中止执行信号 Halt 有效时，则写使能信号则无效。当冲刷信号有效时，则清零信号有效。由于 Logisim 中的寄存器的清零是异步实现，在实现时线路中可能会产生毛刺导致 BUG。因此在流水段清零信号有效时，流水段寄存器需实现同步清零，可通过将输入数据赋值为 0 后写入的方式产生。因此清零信号有效时，写使能信号也必须有效。

从上面分析可以看出每个流水段寄存器需要传递信号是不一样的，流水段寄存器位宽也不一样的。在实验时，可独立设计每个流水段，也可以设计一个通用的流水段寄存器，包含流水段需要传递的所有信号。为了便于调试和验证，需将 PC 和指令 Instr 在流水段寄存器中依次传递。

当 ID 段中的指令控制器生成中止信号 halt 有效时，PC 寄存器和 IF/ID 段寄存器写使能信号无效，其他流水段的指令需继续执行，halt 信号通过流水段寄存器传递到 WB 段后，才能中止程序的执行。

4.5 流水线 CPU 整体设计

为了保证 CPU 每次重启时，都能从相同的初始状态开始执行程序，需要设计一个复位信号 **Reset**，用来初始化 CPU 中各个部件的控制信号，清空数据存储器，设置 PC 寄存器的起始地址 **Init Addr**。为了便于跟踪调试，可设置了断点地址 **BP.PC** 和断点使能信号 **BP.EN**，在断点使能信号有效时，当 PC 与设定断点地址 **BP.PC** 相等时，暂停流水线的执行，观察、调试流水线中指令的执行信息。为了验证程序执行周期数，在设计中添加一个时钟周期计数器，用来记录程序执行的总周期数。

为了测试、验证实验设计，可将各个流水段中 **PC**、**WB.RegWr**、**WB.BusW**、**M.Din**、**M.MemWr**、**BusAFW** 和 **BusBFW** 等信号输出，并且当中止指令 **ecall** 执行结束后，生成 **Stop** 信号，表示程序执行结束。统计程序执行的总周期数、阻塞次数、冲刷次数、转发次数等信息，以便验证冒险处理的可靠性和效率。

在前面单周期 CPU 设计实验已经完成的寄存器堆、ALU、控制器等电路设计，稍加修改后可用在流水线 CPU 设计中，根据流水线 CPU 设计的原理图，实现流水线处理器，并在该处理器上运行 RV32I 的测试程序和官方指令集来验证处理器设计的正确性。

实验步骤如下。

1) 实现流水线 CPU

在 Logisim 中打开实验模版文件 **lab8.1.circ**，在 Logisim 的 **project** 菜单下添加 Logisim 库文件，把实验 3、实验 4 和实验 5 的设计文件添加到 **lab8** 的项目中，将流水线 CPU 设计中需要的功能组件从库文件中的复制到当前工作区中，然后在工作区中放置相应功能组件，并根据需要进行修改，如取指令模块、数据存储器 **RAM**、译码、寄存器堆和控制部件（需修改，当输入 **Stall** 信号有效时，所有输出控制信号清零），以及流水线 CPU 中新增的转发检测、阻塞检测、流水段寄存器、跳转检测等；参考图 8-5 所示的布局引脚。根据模块中定义信号进行连接。设置指令存储器和数据存储器的属性，片选信号都定义为高电平有效。指令存储器的地址位宽设置为 16 位，数据位宽设置为 32 位。

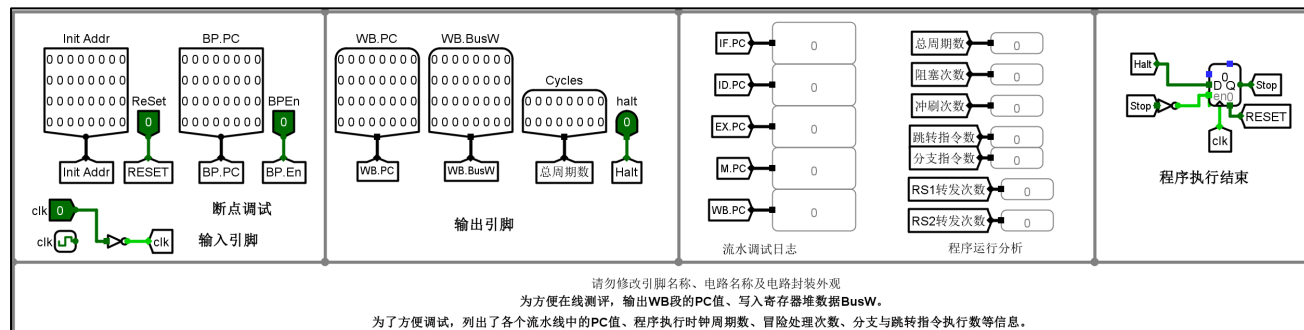


图 8-5 流水线 CPU 引脚布局图

2) 测试指令验证

为了测试系统整体功能和主要指令的执行情况，实验提供了一个汇编语言测试程序 **lab8.1.asm**，用来检

查加减运算、移位、访存、跳转等指令的功能，如果指令执行结果验证通过，则在寄存器堆的 10 号寄存器中写入“00c0ffee”；否则在 10 号寄存器写入“deaddead”，并在 3 号寄存器中写入当前测试指令的序号。

```
.text
main:
test_1: #x0 register,addi,xor,lui,bne test
        lui    x0, 0x80000
        addi   x0, x0,0x7ff
        xor    x1, x1, x1
        addi   x3,x0,1      #R[3]=1
        bne    x0, x1, fail

test_2:   #add test
        lui    x1,0x80000
        lui    x2,0xffff8
        add    x14,x1,x2      #RAW 冒险
        lui    x7,0x7fff8
        addi   x3,x0,2      #R[3]=2
        bne    x14, x7, fail

test_3:   #and test
        lui    x1,0xff01
        addi   x1,x1,-16 # 0ff00ff0
        lui    x2,0xf0f0f
        addi   x2,x2,240 # f0f0f0f0
        and    x15,x1,x2
        lui    x7,0xf00
        addi   x7,x7,0xf0
        addi   x3,x0,3      #R[3]=3
        bne    x15,x7, fail

test_4:   #sll test
        addi   x1,x0,1
        addi   x2,x0,63
        sll    x16,x1,x2
        lui    x7,0x80000
        addi   x3,x0,4      #R[3]=4
        bne    x16,x7, fail

test_5:   #sra test
        lui    x1,0x80000
        addi   x2,x0,14
        sra    x17,x1,x2
        lui    x7,0xfffe0
```

```

        addi x3,x0,5      #R[3]=5
        bne x17,x7, fail

test_6:    #slti test
        lui x1,0x80000
        slti x18,x1,2047
        addi x7,x0,1
        addi x3,x0,6      #R[3]=1
        bne x18,x7, fail

test_7:    #sltiu test
        lui x1,0x80000
        sltiu x19,x1,2047
        addi x7,x0,0
        addi x3,x0,7      #R[3]=7
        bne x19,x7, fail

test_8:    #load,store test
#        la x1,buffer      #等价于下列两条指令
        auipc x1, 2
        addi x1,x1,0xfffff58 #buffer,0x2000
        addi x2,x0,0xee
        sb x2,0(x1)
        addi x2,x0,0xff
        sb x2,1(x1)
        addi x2,x0,0xc0
        sb x2,2(x1)
        addi x2,x0,0x00
        sb x2,3(x1)
        addi x7,x0,-1
        addi x3,x0,8      #R[3]=8
        lb x20,1(x1)
        bne x20,x7, fail  #load-use 冒险
        addi x3,x0,9      #R[3]=9
        lui x7,16 #10
        addi x7,x7,-18 #0x0ffee
        lhu x21,0(x1)
        bne x21,x7, fail  #load-use 冒险
        addi x3,x0,10     #R[3]=10
        lh x22,2(x1)
        addi x7,x0,0xc0
        sh x21,4(x1)
        bne x22,x7, fail  #load-use 冒险
        addi x3,x0,11     #R[3]=11

```

```

    lui    x7,0xc10
    addi x7,x7,-18 # c0ffee
    lw     x23,0(x1)
    bne    x23,x7, fail    #load-use 冒险
    addi x3,x0,12    #R[3]=12
    lui    x7,0xc10
    addi x7,x7,-18 # c0ffee
    addi x4,x1,0
    lw     x24,0(x4)
    sw     x24,4(x4)
    lw     x31,4(x4)
    bne    x31,x7, fail    #load-store 冒险

test_9:    #jalr test
    addi x3,x0,13    #R[3]=13
    auipc  x25, 0
    jalr   x25, x25,16    # test_9_2
test_9_1:
    add x0, x0,x0
    jal x0, fail
test_9_2:
    auipc x7,0
    addi x7,x7,-8    #test_9_1
    bne x25,x7,fail    #auipc,jalr,jal 指令测试及控制冒险处理

test_10:    #loop test, 计算斐波那契项数 n=10
    addi x3,x0,14    #R[3]=14
    addi x8,x0,0x0a    #设置循环次数 n=10
    ori x10,x0,0    #x9=F(0)=0
    ori x11,x0,1    #x10=F(1)=1
    addi x9,x0,1    #x9 初始化计数器
test_10_1:
    add x12,x11,x10    #x12=x11+x10
    add x10,x0,x11
    add x11,x0,x12
    addi x9,x9,1
    bgt x9,x8,test_10_2    #计数器>n,结束循环
    jal test_10_1
test_10_2:
    sw     x12,8(x4)    #保存结果到数据区
pass:
    lui    x10,0xc10
    addi x10,x10,-18 # R[10]=0X00c0ffee
    ecall    #opcode=0x73, 程序中止执行

```

fail:

```
    lui    x10,0xdeade
    addi x10,x10,-339 # R[10]=0Xdeaddead
    ecall                    #opcode=0x73, 程序中执行
.data    # 在 RARS 中可设置数据段从 0x2000 开始
buffer: .word 100
```

可以在 RARS 中调试该程序 lab8.1.asm，并导出 16 进制程序代码。为了简化实验步骤，本次实验提供了生成的机器代码数据文件 lab8.1.hex，在指令存储器中加载 lab8.1.hex 文件，按照时钟单步执行，观测实验结果，写出 3 号、10 号和 12 号寄存器和数据区 buffer[0-2]的内容，验证电路功能、分支次数、阻塞次数和转发次数等性能参数，保存电路文件为 lab8.1.circ，提交到在线实验平台进行评测。

2、测试程序优化验证

在流水线 CPU 与单周期 CPU 设计中一样，指令存储器与数据存储器分离。因此可以使用累加和测试程序来验证，并通过设置参数，分析测试程序指令执行情况与输出的时钟周期总数、阻塞次数、冲刷次数、转发次数的统计数据对比验证。

计算累加和的测试程序参考设计如下：

计算累加和程序 lab8.2.asm

```
.text
main:
    lw a0,0(x0)          # 从数据区地址单元 0x0 中读取参数 n 到寄存器 a0; n=0,则退出
    beq a0,x0,fail
    ori a2,x0,1          # 循环变量 i, 存放在 a2, 初值为 1
    xor a3,a3,a3         # 累计和存放在 a3, 初值为 s=0
loop:
    add a3, a3, a2        # 将 a3=a3+i
    bgeu a2, a0, finish  # 若 i>=n, 则跳出循环
    addi a2, a2, 1        # i++
    jal x0, loop          # 无条件跳转到 loop 执行
finish:
    sw a3, 4(x0)          # 将累加结果保存到数据存储器地址 0x2004 单元

pass:
    lui    a0,0xc10
    addi   a0,a0,-18      # a0=00c0ffee
    ecall                    # opcode=0x73, 程序中执行

fail:
    lui    a0,0xdeade
    addi   a0,a0,-339     # a0=deaddead
    ecall                    # opcode=0x73, 程序中执行
```

在 RARS 工具编译调试该汇编程序，为了模拟 CPU 的系统状态中，需配置 RARS 的内存分配模式，在

菜单 Setting 中的 Memory Configuration 选项设置为 “Compact, Data at Address 0”，这样数据段的起始地址从 0 开始；在数据段中修改参数 n，验证程序正确性后，通过 Dump to File 导出代码段，将程序可执行机器代码以十六进制数据文本格式导出到 lab8.2.hex 文件中，在该文件首行前插入文本 “v2.0 raw” 后保存。

在流水线 CPU 的指令存储器（ROM）中加载测试程序二进制代码文件 lab8.2.hex，然后在数据存储器 DataRAM 地址单元 0x0 中设置初始参数 0x64，启动时钟信号，执行程序，观察输出结果。性能输出如图 8-6 所示。

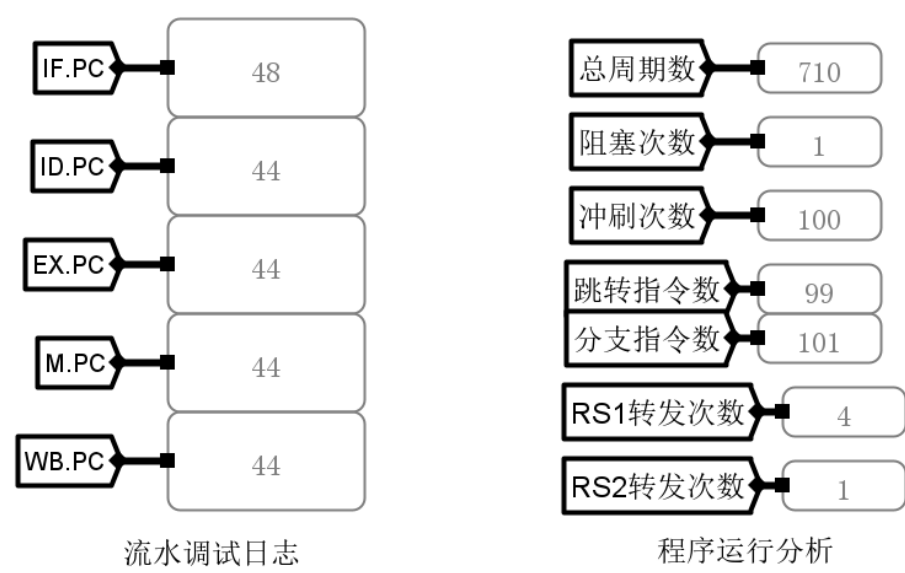


图 8-6 累加和程序执行性能

分析汇编程序，思考如何降低分支指令的执行次数，减少指令执行总周期数。修改 lab8.2.asm 程序，优化程序代码，修改分支语句判断条件，删除无条件跳转语句 “jal x0,loop”，参考设计如下：

```
#计算累加和程序 lab8.2-1.asm
.text
main:
    lw a0,0(x0)          # 从数据区地址单元 0x0 中读取参数 n 到寄存器 a0; n=0,则退出
    beq a0,x0,fail
    ori a2,x0,1          # 循环变量 i，存放在 a2，初值为 1
    xor a3,a3,a3         # 累计和存放在 a3，初值为 s=0
loop:
    add a3, a3, a2        # 将 a3=a3+i
    addi a2, a2, 1        # i++
    bgeu a0,a2,loop       # 若 n>=i，则循环
    sw a3, 4(x0)          # 将累加结果保存到数据存储器地址 0x2004 单元

pass:
    lui    a0,0xc10
    addi   a0,a0,-18      # a0=00c0ffee
    ecall                                # opcode=0x73，程序中执行

fail:
```

```

lui    a0,0xdeade
addi   a0,a0,-339    # a0=deaddead
ecall                      # opcode=0x73, 程序中中止执行

```

在 RARS 工具编译调试该汇编程序 lab8.2-1.asm，在数据段中修改参数 n，验证程序正确性后，通过 Dump to File 导出代码段，将程序可执行机器代码以十六进制数据文本格式导出到 lab8.2-1.hex 文件中，在该文件首行前插入文本“v2.0 raw”后保存。在流水线 CPU 的指令存储器中加载测试程序二进制代码文件 lab8.2.hex，然后在数据存储器 DataRAM 地址单元 0x0 中设置初始参数 0x64，启动时钟信号，执行程序，观察程序优化后的输出结果如图 8-7 所示。

重新编译后，进行测试，验证设计思路。

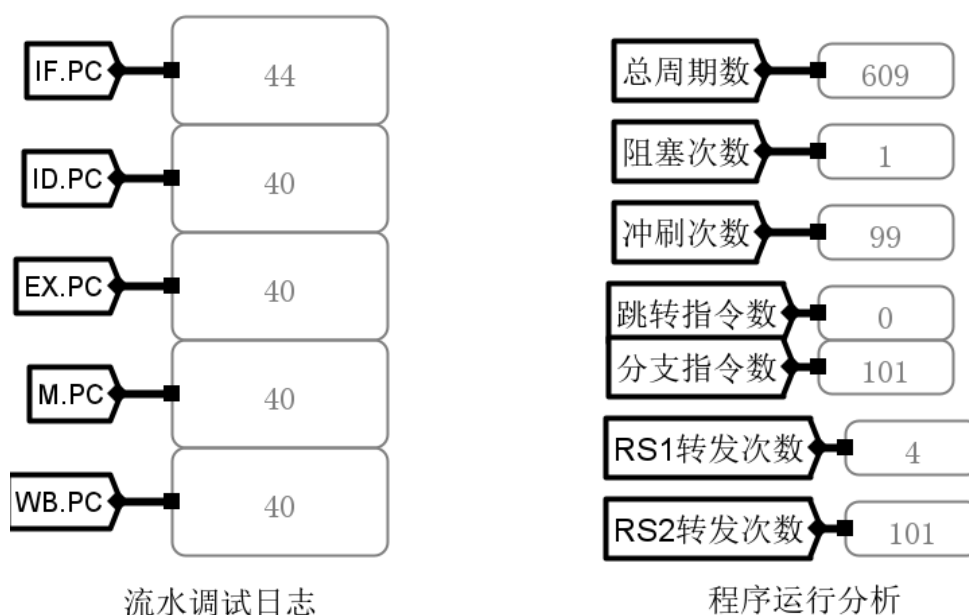


图 8-7 累加和程序优化后的执行性能

在指令存储器中加载 lab8.2.tst.hex 文件，保存电路设计文件为 lab8.2.circ，然后提交到在线实验平台上进行评测。

3、结构优化设计

在前面实验设计中，在 M 段执行跳转检测模块，当分支预测失败后需要冲刷前面 IF、ID、EX 流水段中指令信息，时间片为 3。如果将跳转检测模块迁移到前面的流水段执行，则可以减少跳转是需要冲刷的指令数。在 ID 段生成了指令类型、立即数，并读取了寄存器堆中 rs1 和 rs2 寄存器的内容，但是源操作数有可能需要转发，需要在 EX 段才能确定，可在 EX 段执行跳转检测。

修改流水线 CPU 的设计电路，将跳转检测模块迁移到 EX 段，保存电路设计文件为 lab8.3.circ。在指令存储器中加载 lab8.2-1.hex 文件，在数据存储器地址单元 0x0 中设置初始参数 0x64，执行程序，观察结果如图 8-8 所示，总周期数减少到 510 个，并分析原因。

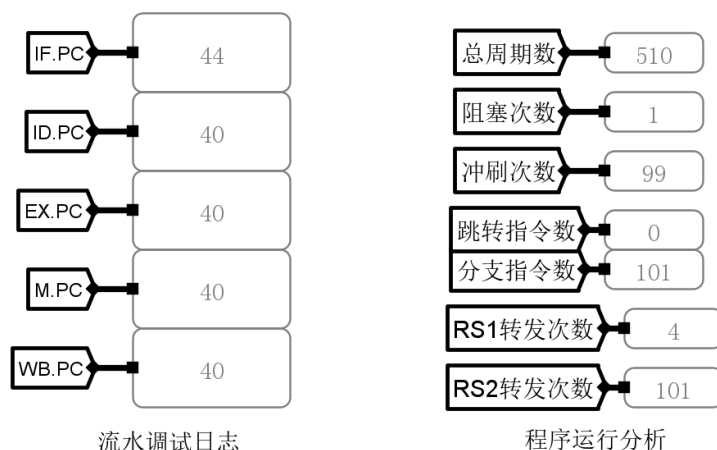


图 8-8 设计优化后的累加和程序执行性能

可以使用冒泡排序程序来验证，并通过设置参数，分析测试程序指令执行情况与输出的时钟周期总数、阻塞次数、冲刷次数、转发次数的统计数据进行对比验证。

冒泡程序 lab8.3.asm 参考设计如下：

```
.text
main:
    lw    t0, 0(x0)        # t0 = n (元素个数)
    addi  a1, x0, 1        # a1 = 1 (常量)
    addi  a0, x0, 4        # a0 = 数组起始地址(0x4)
    bge   a1, t0, exit     # 处理特殊情况 (n<=1),直接退出
    add   a2, t0, x0       # i = n (外层循环计数器)

outer_loop:
    addi  a3, x0, 0        # j = 0 (内层循环计数器)
    sub   t1, a2, a1       # t1 = i-1 (内层循环上限)

inner_loop:
    slli  a4, a3, 2        # 计算元素偏移地址
    add   a4, a4, a0       # a4 = &a[j]=a0+j*4
    lw    a6, 0(a4)        # a6 = a[j]
    lw    a7, 4(a4)        # a7 = a[j+1]
    bge   a7, a6, no_swap  # 若 a6<=a7 有序,则不交换
    sw    a7, 0(a4)        # 交换元素
    sw    a6, 4(a4)

no_swap:
    addi  a3, a3, 1        # j++
    blt   a3, t1, inner_loop # 继续内层循环(j < i-1)
    sub   a2, a2, a1       # i--
    bne   a2, a1, outer_loop # 继续外层循环(i > 1)

exit:
    ecall
```

在 RARS 工具编译调试该汇编程序 lab8.3.asm，在数据段中修改待排序参数（待排序数目及待排序数），验证程序正确性后，通过 Dump to File 导出代码段，将程序可执行代码以十六进制数据文本格式导出到 lab8.3.hex 文件中，在该文件首行前插入文本“v2.0 raw”后保存。在 lab8.3.circ 的指令存储器中加载

测试程序文件 lab8.3.hex，然后在数据存储器 RAM3~RAM0 中分别加载数据镜像文件 lab8.3.ram3.txt~lab8.3.ram0.txt，启动时钟信号，执行程序，观察数据存储器中排序结果，并验证程序性能，程序性能的输出结果如图 8-9 所示。

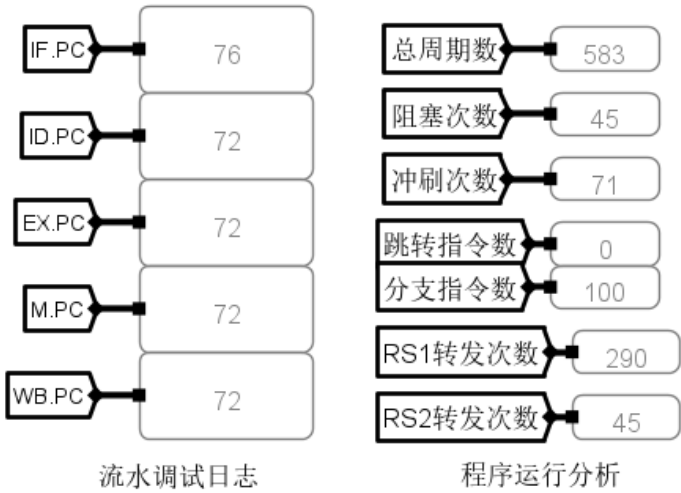


图 8-9 冒泡排序测试程序执行性能

可以优化冒泡排序程序或设计其他排序算法，进行程序性能比较，找出在该 CPU 设计下执行性能最好的排序算法。

在指令存储器中加载 lab8.3.tst.hex 文件，保存电路设计文件，然后提交到在线实验平台上进行评测。

4、官方测试验证

打开实验 8 提供的官方测试集文件 testcase.zip，在流水线 CPU 中加载每条指令的测试文件，验证实验结果，保存程序执行结果。

需要提醒的是在指令测试代码中判断指令测试成功或失败的结束代码是“dead10cc”，需根据该代码来中止程序的执行，在控制器中增加中止信号生成逻辑，修改后的流水线 CPU 设计电路另存为 lab8.4.circ。

在 lab8.4.circ 的指令存储器中加载指令测试代码后，选择连续时钟信号，执行程序。如果指令测试通过，则在 a0 寄存器中的数据为 0x00c0ffee，如果测试不通过，则 a0 寄存器中的数据为 0xdeaddead。

指令测试通过后，依次填写官方测试集验证数据表，如表 8.1 所示。

表 8.1 RSICV 官方测试集测试数据表

序号	测试指令	时钟周期总数	阻塞次数	冲刷次数	跳转指令 执行数	分支指令 执行数	rs1 转发次数	rs2 转发次数
1	add							
2	addi							
3	and							
4	andi							
5	auipc							
6	beq							
7	bge							

8	bgeu							
9	blt							
10	bltu							
11	bne							
12	jal							
13	jalr							
14	lb							
15	lbu							
16	lh							
17	lhu							
18	lui							
19	lw							
20	or							
21	ori							
22	sb							
23	sh							
24	sll							
25	slli							
26	slt							
27	slti							
28	sltiu							
29	sltu							
30	sra							
31	srai							
32	srl							
33	srli							
34	sub							
35	sw							
36	xor							
37	xori							

五、思考题

- 1、转发检测模块中，没有根据操作码 OP 来判断当前指令是否操作 rs1 和 rs2 字段，可能对输出产生什么影响？会导致程序执行错误吗？如何解决该问题，是否必要？被阻塞的指令能产生有效转发信号吗？
- 2、本实验中将跳转检测放在 EX 段执行，能否继续优化迁移到 ID 段执行？试分析可行性。
- 3、根据基础流水线 CPU 中累加和测试程序三种情形的执行性能，分别讨论总周期数、阻塞次数、rs1 转发次数、rs2 转发次数、分支和跳转指令执行次数数据准确性，计算程序的平均 CPI，并讨论提升该测试程序执行效率的方法。
- 4、如何修改流水线 CPU 的设计，实现非法指令和结果溢出两种异常处理，标识异常类型，跳转异常处理程序。
- 5、在实验中对于分支指令采用静态预测的方法来解决控制冒险，讨论如何实现 1 位动态预测的方法。
- 6、简述高级流水线设计方法。